LAPORAN PRAKTIKUM

POSTTEST 4

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

RENDY (2509106069)

Kelas (B2 ‘25)

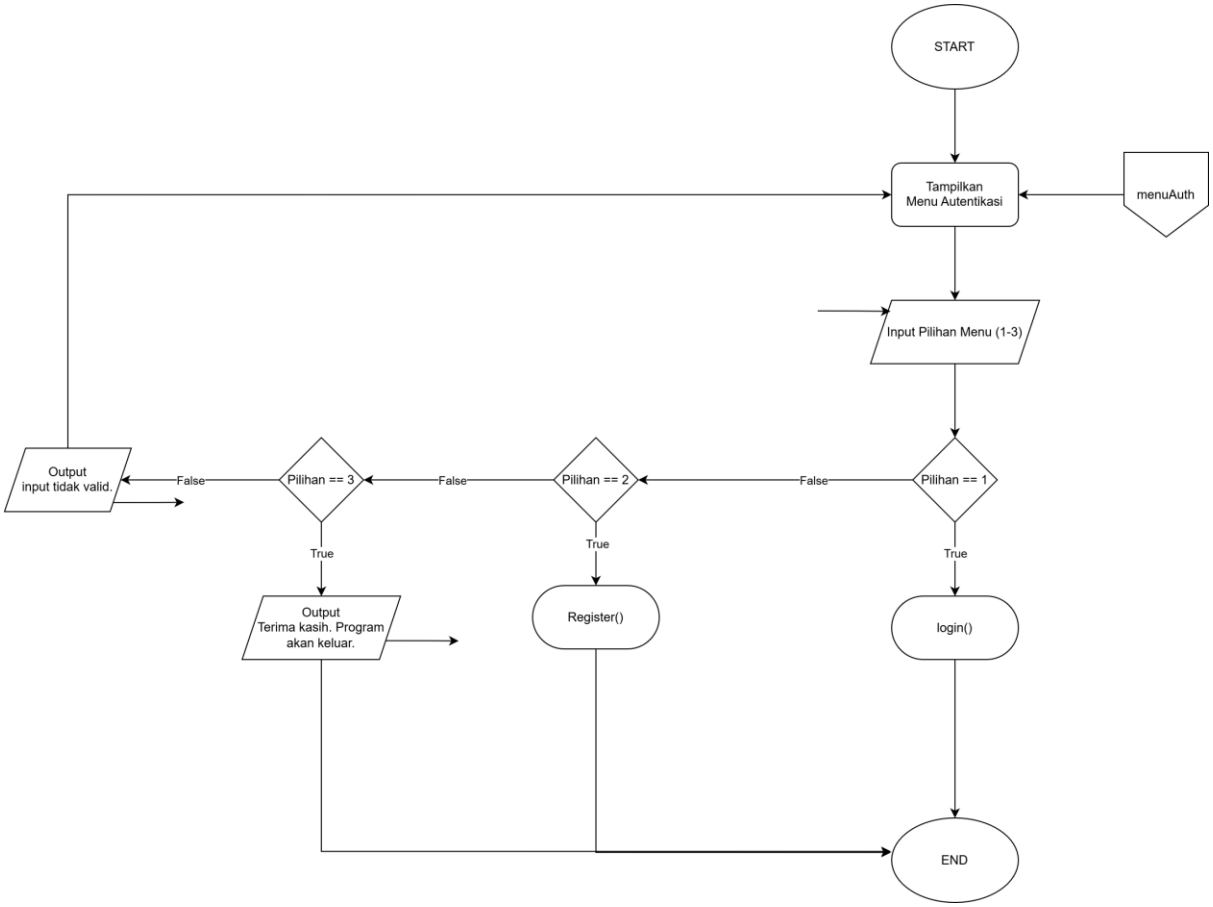
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

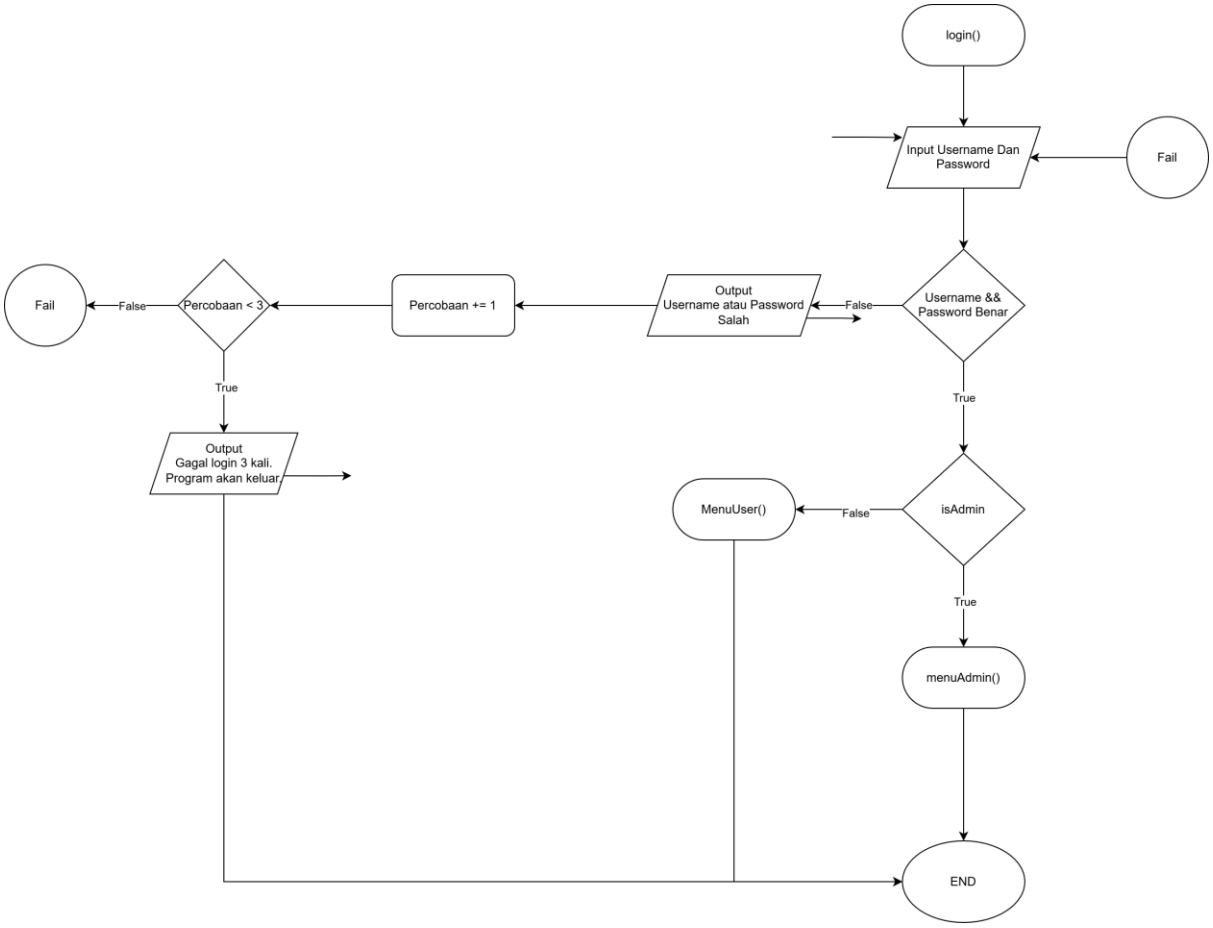
SAMARINDA

2026

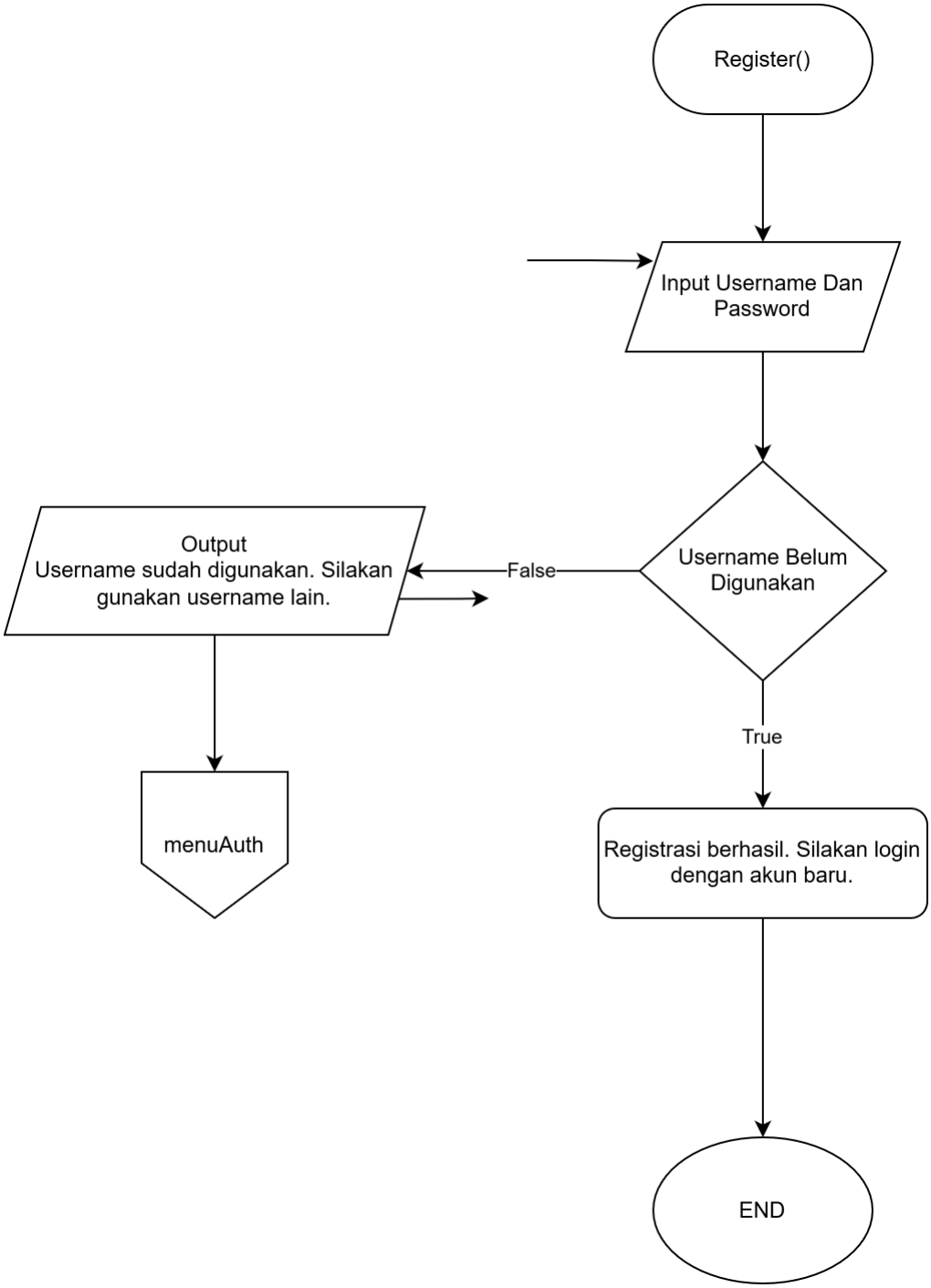
1. Flowchart



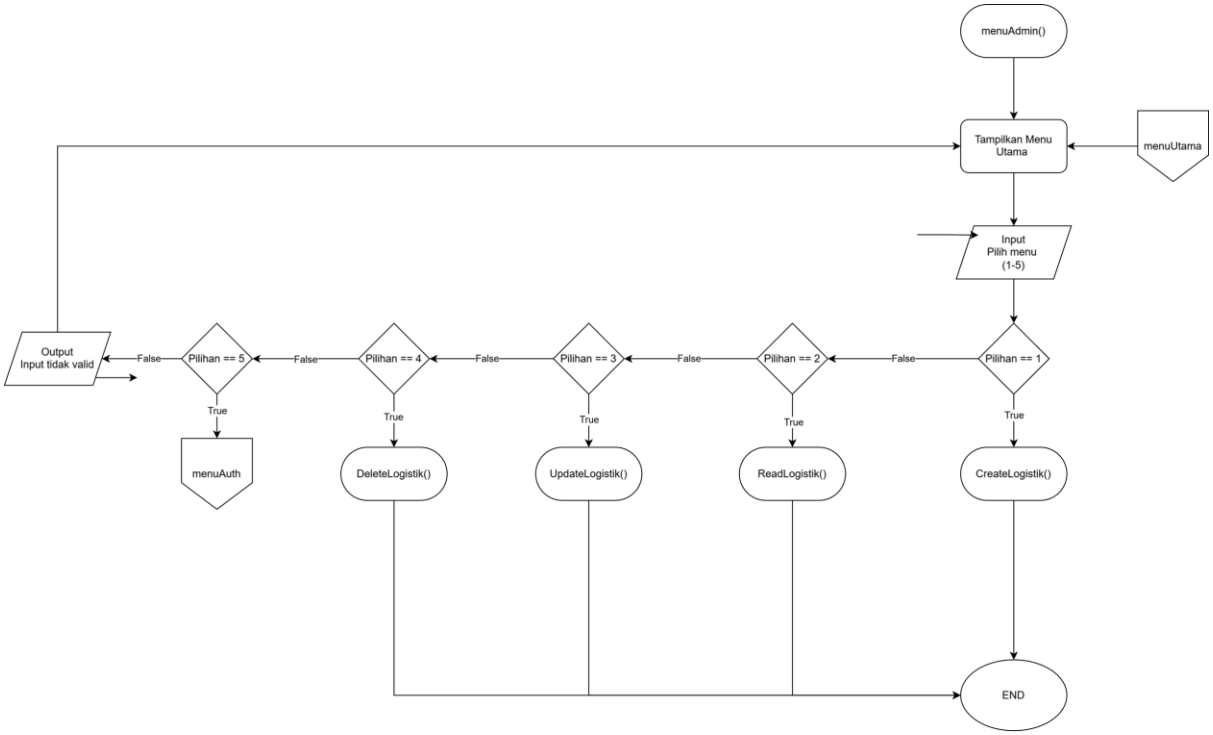
Gambar 1.1 Flowchart Menu Auth



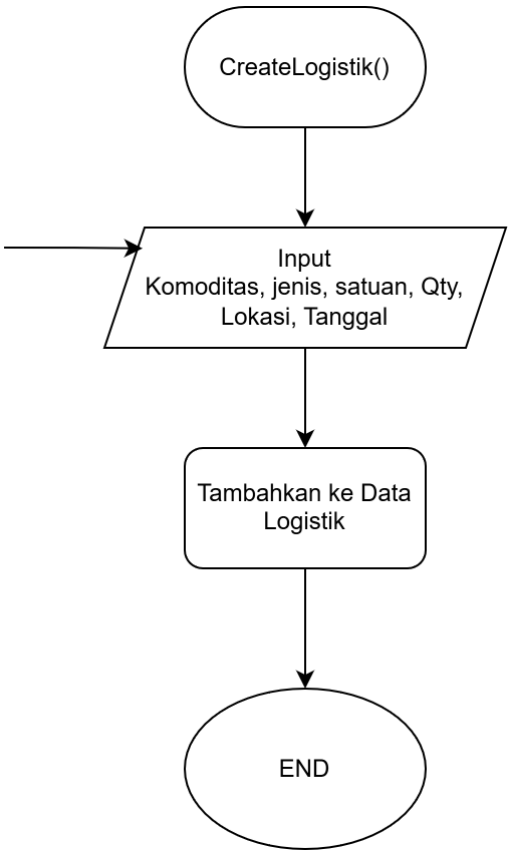
Gambar 1.2 Flowchart Login



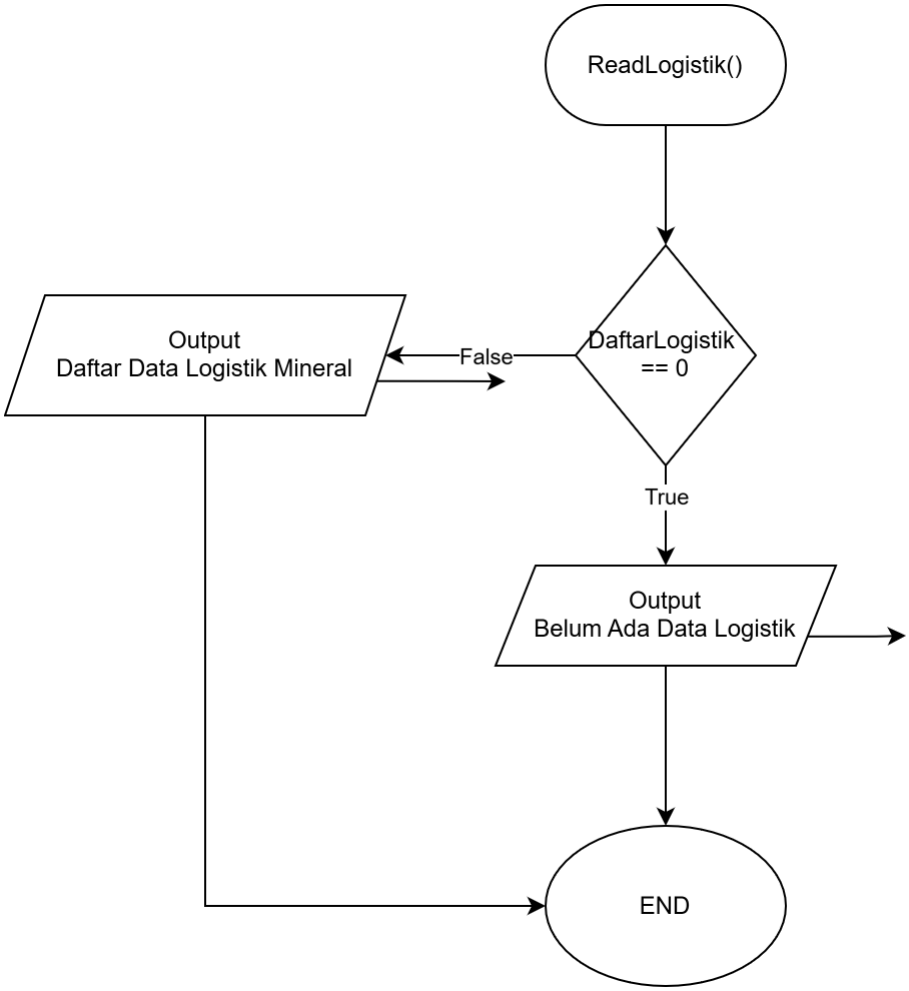
Gambar 1.3 Flowchart register



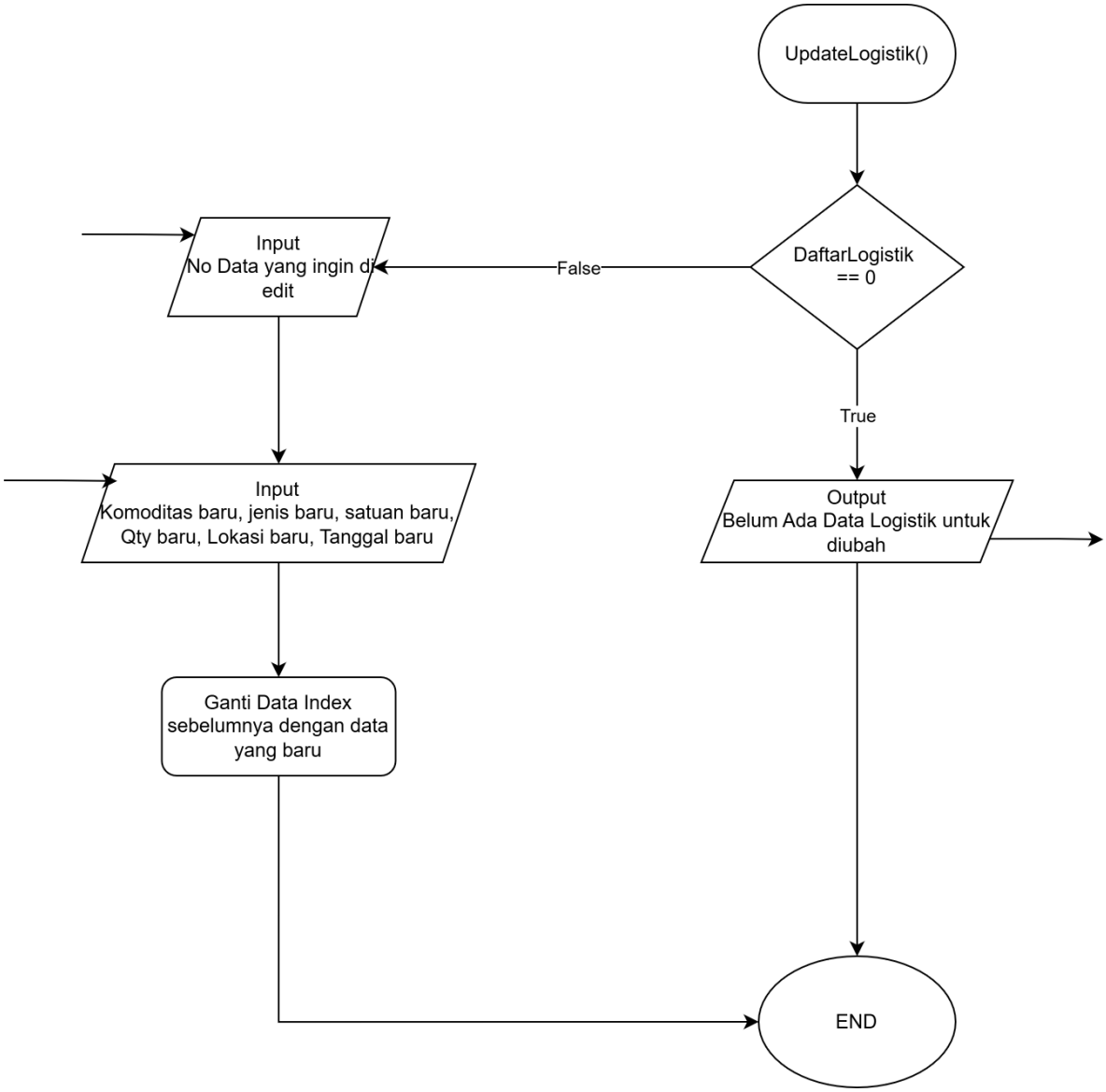
Gambar 1.4 Flowchart menu Admin



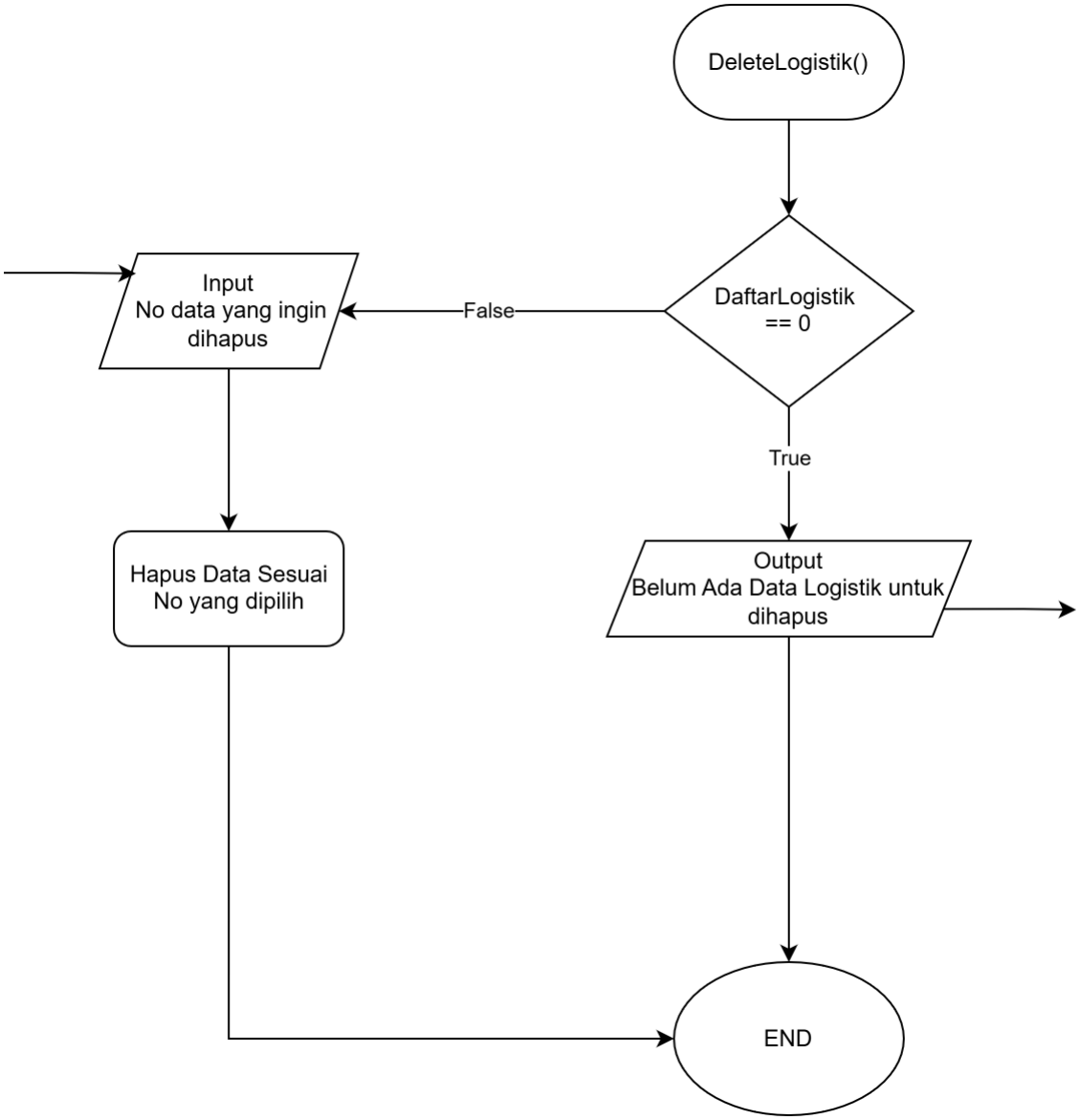
Gambar 1.5 Flowchart logic Create



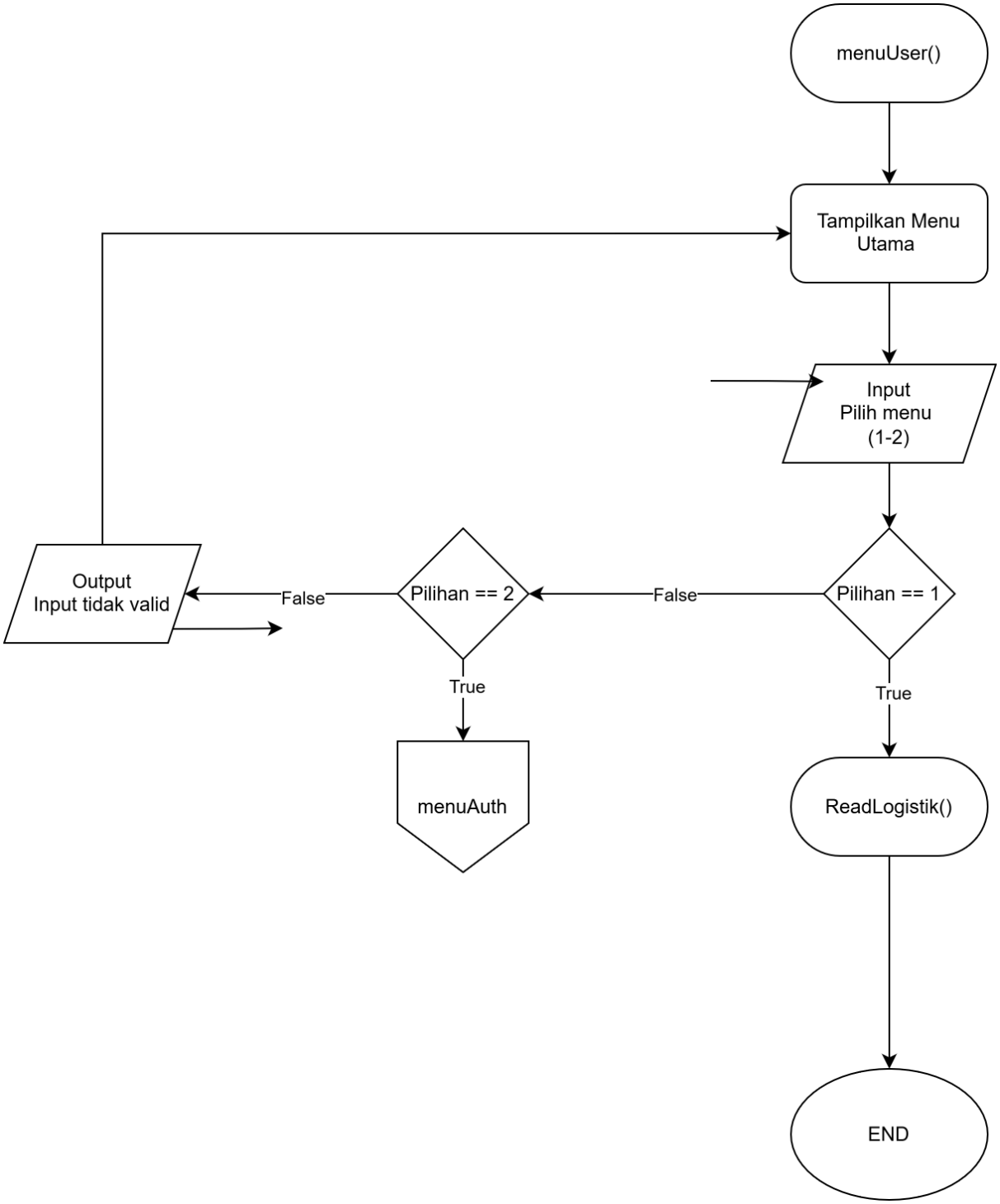
Gambar 1.6 Flowchart Logic Read



Gambar 1.7 Flowchart Logic Update



Gambar 1.8 Flowchart Logic Delete



Gambar 1.9 Flowchart Menu User

Alur Logika Program Sistem Logistik Pertambangan Mineral

1. Autentikasi (Login & Register)

Program dimulai dengan menu autentikasi untuk memastikan keamanan akses data.

- Menu Autentikasi: pengguna diberikan 3 pilihan utama: Login, Register, atau keluar.
- Register: jika memilih opsi 2, pengguna diminta memasukkan *Username* dan *password.*
- Login:
  1. Sistem meminta input *Username* dan *Password*
  2. Validasi: sistem mencocokkan input dengan data yang disimpan di array users.
  3. Keamanan: jika login gagal, sistem menghitung jumlah percobaan.pengguna memiliki maksimal 3 kali kesempatan.jika gagal melampaui batas tersebut, program akan berhenti otomatis (fail).
  4. Berhasil: jika data cocok, sistem mengindentifikasi apakah pengguna adalah Admin atau User biasa, lalu lanjut ke Menu Utama.

2. Menu Utama Dan Fitur (CRUD)

Setelah login Sukses, pengguna masuk ke menu utama dengan hak akses yang berbeda:

- Menu Admin (Full Access):
  1. Tambah Data (Create): menginput data mineral baru. ID Logistik (Misal: LOG001) digenerate secara otomatis.
  2. Lihat Data (Read): Menampilkan Seluruh daftar logistik dalam bentuk tabel yang rapi.
  3. Ubah Data (Update): Memilih data spesifik berdasarkan nomor urut untuk memperbarui informasi komoditas, jenis, kuantitas,dll.
  4. Hapus Data (Delete): Menghapus data Logistik dari Sistem.
  5. Logout: Kembali ke meu Autentikasi Awal.
- Menu User (limited Access):
  1. Lihat Data (Read): Hanya dapat melihat daftar logistik tanpa bisa menambah, mengubah, atau menghapus.
  2. Logout: Kembali ke menu autentikasi Awal.

3. Output dan Perulangan

- Validasi Input: Setiap input menu divalidasi agar hanya menerima angka yang sesuai (misal 1 – 5 untuk admin). Jika input salah atau berupa huruf, sistem menampilkan pesan error dan meminta input ulang.
- Sistem looping: program menggunakan perulangan while (true), sehingga setelah selesai melakukan satu aksi (seperti menambahkan data), pengguna akan dibawa kembali ke Menu Utama.
- Pengakhiran Program: Program hanya akan benar benar berhenti jika pengguna memilih opsi “Keluar program” pada menu autentikasi awal (opsi 3)

2. Deskripsi Singkat Program

Program ini mewajibkan pengguna untuk melakukan login terlebih dahulu dengan memasukkan username dan password berupa 3 digit terakhir NIM. Program ini juga mempunyai fitur login yang membatasi pengguna untuk melakukan maksimal 3 kali percobaan; jika pengguna salah memasukkan password sebanyak 3 kali, maka program otomatis akan berhenti.

Setelah login berhasil, program menyediakan menu utama untuk mengelola data logistik mineral yang mencakup fitur Create (tambah data), Read (lihat daftar data), Update (ubah data), dan Delete (hapus data). Pengguna dapat memilih jenis fitur yang diinginkan, mengelola detail informasi seperti nama mineral, kuantitas, serta asal tambang, dan program akan menampilkan data tersebut secara terperinci sebelum memberikan pilihan untuk keluar ke menu autentikasi atau menutup aplikasi.

3. Source Code

A. Deklarasi Variabel

Mendeklarasi variable seperti struct, array, palette warna, dll

Source Code:

```
#include <iostream>
#include <iomanip>

#ifdef _WIN32
    #define CLEAR system("cls") // bersihkan terminal (windows)
    #include <windows.h> // untuk ascii art dan warna (windows)
#else
    #define CLEAR system("clear") // bersihkan terminal (linux, macOs)
#endif

// palette warna
#define RESET       "\033[0m"
#define HIJAU       "\033[38;5;82m"
#define COKELAT     "\033[38;5;94m"
#define ABU_TERANG  "\033[38;5;246m"
#define UNGU_GELAP  "\033[38;5;55m"
#define BIRU        "\033[38;5;87m"
#define EMAS        "\033[38;5;220m"
#define MERAH       "\033[38;5;196m"
#define TOSKA       "\033[38;5;48m"
#define PUTIH       "\033[97m"
#define KUNING      "\033[93m"
#define UNGU        "\033[38;5;129m"
#define ABU_REDUP   "\033[90m"
```

Gambar 3.1 Deklarasi Preprocessor & palette warna

```
struct User {
    string username;
    string password;
    bool isAdmin;
};

struct Logistik {
    string idLogistik;
    struct DetailMineral {
        string namaMineral;
        string jenisMineral;
        double kuantitas;
        string satuan;
    } mineral;

    struct Alamat {
        string asalTambang;
    } asal;
    string tanggal;
    string status;
};

const int MAX_USERS = 100;
const int MAX_LOGISTIK = 100;
```

Gambar 3.2 Deklarasi Variabel & Struct

B. Fitur Menu Login (Auth)

Fitur ini akan menampilkan menu login yang berisi pilihan login, registrasi, dan keluar dari Program

Source Code:

```
void displayMenuAuth() {
    cout << ABU_REDUP << "\n\n=========================================================" << endl;
    cout << EMAS   <<    "|           SISTEM LOGISTIK MINERAL PERTAMBANGAN        |" << endl;
    cout << ABU_REDUP << "=========================================================" << endl;
    cout << PUTIH  <<    "| No | Autentikasi                                      |" << endl;
    cout << ABU_TERANG <<"|----|--------------------------------------------------|" << endl;
    cout << PUTIH  <<    "| 1. | Login Akun                                       |" << endl;
    cout << PUTIH  <<    "| 2. | Register Akun Baru                               |" << endl;
    cout << PUTIH  <<    "| 3. | Keluar Program                                   |" << endl;
    cout << ABU_REDUP << "=========================================================" << RESET << endl;
}
```

Gambar 3.3 Menu Auth

C. Login User

Memverifikasi Username dan password, jika username dan password benar, maka akan masuk ke dalam menu utama, serta membatasi percobaan login maksimal 3 kali

Source Code:

```
void displayMenuAuth() {
    cout << ABU_REDUP << "\n\n=========================================================" << endl;
    cout << EMAS   <<    "|           SISTEM LOGISTIK MINERAL PERTAMBANGAN        |" << endl;
    cout << ABU_REDUP << "=========================================================" << endl;
    cout << PUTIH  <<    "| No | Autentikasi                                      |" << endl;
    cout << ABU_TERANG <<"|----|--------------------------------------------------|" << endl;
    cout << PUTIH  <<    "| 1. | Login Akun                                       |" << endl;
    cout << PUTIH  <<    "| 2. | Register Akun Baru                               |" << endl;
    cout << PUTIH  <<    "| 3. | Keluar Program                                   |" << endl;
    cout << ABU_REDUP << "=========================================================" << RESET << endl;
}
```

Gambar 3.4 Menu Auth

```
int login(User *users, int *jumlahUser) {
    if (*jumlahUser == 0) {
        CLEAR;
        cout << "==========================================================" << endl;
        cout << "         Sistem Logistik Mineral Pertambangan          " << endl;
        cout << "                           LOGIN" << endl;
        cout << "==========================================================" << endl;
        cout << "[i] " << "Belum ada user terdaftar. Silakan register dulu." << endl;
        tekanEnter();
        return -2;
    }

    int idxLogin = -1;
    for (int percobaan = 1; percobaan <= 3; percobaan++) {
        CLEAR;
        cout << ABU_REDUP << "=========================================" << endl;
        cout << EMAS   << "||            LOGIN SESSION  # " << percobaan << "          ||" << endl;
        cout << ABU_REDUP << "=========================================" << endl;

        string username, password;

        while (true) {
            cout << PUTIH << "\n  [>] Masukkan Username: " << BIRU;
            getline(cin, username);
            cout << RESET;
            if (!username.empty()) break;
            cout << MERAH << "[!] " << PUTIH << "Username tidak boleh kosong." << RESET << endl;
        }
        while (true) {
            cout << PUTIH << "  [>] Masuk Password: " << BIRU;
            getline(cin, password);
            cout << RESET;
            if (!password.empty()) break;
            cout << MERAH << "[!] " << PUTIH << "Password tidak boleh kosong." << RESET << endl;
        }
```

Gambar 3.5 Fungsi Login

```
for (int i = 0; i < *jumlahUser; i++) {
            User *ptrUser = users + i; // pointer arithmetic
            if (ptrUser->username == username && ptrUser->password == password) {
                idxLogin = i;
                break;
            }
        }

        if (idxLogin != -1) {
            User *ptrLogin = users + idxLogin; // pointer ke user yang login
            cout << HIJAU << "\n[OK] " << PUTIH << "Login berhasil. Selamat datang, " << BIRU << ptrLogin->username << PUTIH << "." << RESET << endl;
            tekanEnter();
            return idxLogin;
        }

        cout << MERAH << "\n[!] " << PUTIH << "Username atau Password salah." << RESET << endl;
        if (percobaan < 3) {
            tekanEnter();
        }
    }

    cout << MERAH << "\n[!] BATAS PERCOBAAN HABIS " << PUTIH << "Program akan keluar." << RESET << endl;
    return -1;
}
```

Gambar 3.6 fungsi login

D. Registrasi User

Digunakan untuk Menambahkan akun user baru kedalam array users jika username belum pernah digunakan

Source Code:

```
bool registerUser(User *users, int *jumlahUser) {
    CLEAR;
    cout << ABU_REDUP << "============================================================" << endl;
    cout << EMAS   << "         Sistem Logistik Mineral Pertambangan          " << endl;
    cout << PUTIH  << "                        REGISTER" << endl;
    cout << ABU_REDUP << "============================================================" << RESET << endl;

    if (*jumlahUser >= MAX_USERS) {
        cout << MERAH << "[!] " << PUTIH << "Kapasitas user penuh. Tidak bisa mendaftar user baru." << RESET << endl;
        tekanEnter();
        return false;
    }

    // Pointer ke slot user baru menggunakan pointer arithmetic
    User *ptrUserBaru = users + (*jumlahUser);

    string username, password;

    while (true) {
        cout << PUTIH << "Username: " << BIRU;
        getline(cin, username);
        cout << RESET;
        if (!username.empty()) {
            // Cek apakah username sudah ada
            bool usernameAda = false;
            for (int i = 0; i < *jumlahUser; i++) {

                User *ptrCek = users + i; // pointer arithmetic
                if (ptrCek->username == username) {
                    usernameAda = true;
                    break;
                }
            }
```

Gambar 3.7 fungsi registrasi

```

        if (usernameAda) {
            cout << MERAH << "[!] " << PUTIH << "Username sudah digunakan. Silakan gunakan Username lain." << RESET << endl;
        } else {
            break;
        }
    } else {
        cout << MERAH << "[!] " << PUTIH << "Username tidak boleh kosong." << RESET << endl;
    }
}
```

Gambar 3.8 fungsi registrasi

```
while (true) {
        cout << PUTIH << "Password: " << BIRU;
        getline(cin, password);
        cout << RESET;
        if (!password.empty()) break;
        cout << MERAH << "[!] " << PUTIH << "Password tidak boleh kosong." << RESET << endl;
    }

    // Mengisi data melalui pointer ke struct
    ptrUserBaru->username = username;
    ptrUserBaru->password = password;
    ptrUserBaru->isAdmin = false;

    // dereference pointer lalu increment
    (*jumlahUser)++;

    cout << HIJAU << "\n[OK] " << PUTIH << "Registrasi berhasil. Silakan login dengan akun baru." << RESET << endl;
    tekanEnter();
    return true;
}
```

Gambar 3.9 fungsi registrasi

E. Menambah Data Logistrik

Digunakan untuk menambahkan data logistik (Create)

Source Code:

```
void create(Logistik *logistik, int *jumlahLogistik) {
    CLEAR;
    cout << ABU_REDUP << "==========================================================" << endl;
    cout << ABU_TERANG << "--- [" << BIRU << " TAMBAH DATA " << ABU_TERANG << "] ---" << RESET << endl;
    cout << ABU_REDUP << "==========================================================" << endl;

    if (*jumlahLogistik < MAX_LOGISTIK) {

        Logistik *ptrBaru = logistik + (*jumlahLogistik);

        // Auto-generate ID secara rekursif
        ptrBaru->idLogistik = generateIdRekursif(logistik, jumlahLogistik);

        cout << PUTIH << "ID Logistik           : " << TOSKA << ptrBaru->idLogistik << ABU_TERANG << " (otomatis)" << RESET << endl;

        cout << PUTIH << "Komoditas             : " << KUNING;
        getline(cin, ptrBaru->mineral.namaMineral);

        cout << PUTIH << "Jenis                 : " << KUNING;
        getline(cin, ptrBaru->mineral.jenisMineral);

        cout << PUTIH << "Satuan                : " << KUNING;
        getline(cin, ptrBaru->mineral.satuan);

        cout << PUTIH << "Qty                   : " << KUNING;
        cin >> ptrBaru->mineral.kuantitas;
        cin.ignore(1000, '\n');

        cout << PUTIH << "Lokasi                : " << KUNING;
        getline(cin, ptrBaru->asal.asalTambang);

        cout << PUTIH << "Tanggal (dd-mm-yyyy)  : " << KUNING;
        getline(cin, ptrBaru->tanggal);
        cout << RESET;
```

Gambar 3.10 logic create

```
// Pilih status
        cout << "\n" << PUTIH << "    Status Pengiriman:" << endl;
        cout << KUNING << "    [1]  Pending" << endl;
        cout << BIRU <<   "    [2]  Shipped" << endl;
        cout << HIJAU <<  "    [3] Delivered" << RESET << endl;

        int statusPilihan = inputMenu(1, 3, "Pilih Status: ");
        switch (statusPilihan) {
            case 1: ptrBaru->status = "Pending"; break;
            case 2: ptrBaru->status = "Shipped"; break;
            case 3: ptrBaru->status = "Delivered"; break;
        }

        // dereference pointer lalu increment
        (*jumlahLogistik)++;

        cout << HIJAU << "\n[OK] " << PUTIH << "Data berhasil ditambahkan!" << RESET << endl;
    } else {
        cout << MERAH << "[!] " << PUTIH << "Kapasitas data logistik penuh!" << RESET << endl;
    }

    cout << ABU_TERANG << "\nTekan Enter untuk melanjutkan..." << RESET;
    cin.ignore(1000, '\n');
}
```

Gambar 3.11 logic create

F. Melihat Daftar logistik

Menampilkan seluruh daftar logistik yang telah ditambahkan dalam bentuk Tabel.

Source Code:

```
void read(Logistik *logistik, int *jumlahLogistik) {
    CLEAR;
    cout << ABU_REDUP << "\n=========================================================" << endl;
    cout << EMAS      << "|                DAFTAR DATA LOGISTIK MINERAL            |" << endl;
    cout << ABU_REDUP << "=========================================================" << RESET << endl;

    if (*jumlahLogistik == 0) {
        cout << MERAH << "[i] " << PUTIH << "Belum ada Logistik." << RESET << endl;
    } else {
        displayDataLogistik(logistik, jumlahLogistik);
    }

    tekanEnter();
}
```

Gambar 3.12 logic Read

G. Mengubah Data logistik

Menampilkan semua daftar data logistik terlebih dahulu, lalu memperbarui data sesuai indeks yang dipilih user.

Source Code:

```
void update(Logistik *logistik, int *jumlahLogistik) {
    CLEAR;
    cout << ABU_REDUP << "\n=========================================================" << endl;
    cout << EMAS      << "|                UPDATE DATA LOGISTIK MINERAL            |" << endl;
    cout << ABU_REDUP << "=========================================================" << RESET << endl;

    if (*jumlahLogistik == 0) {
        cout << MERAH << "[!] " << PUTIH << "Belum ada data logistik untuk diubah." << RESET << endl;
        tekanEnter();
        return;
    }

    cout << ABU_TERANG << "--- [" << BIRU << " UBAH DATA " << ABU_TERANG << "] ---" << RESET << endl;
    cout << ABU_REDUP << "====================" << RESET << endl;

    for (int i = 0; i < *jumlahLogistik; i++) {
        Logistik *ptr = logistik + i;
        cout << PUTIH << "Data ke-" << EMAS << i + 1 << endl;
        cout << PUTIH << "ID: " << BIRU << ptr->idLogistik << endl;
        cout << PUTIH << "Komoditas: " << KUNING << ptr->mineral.namaMineral << RESET << endl;
        cout << endl;
    }

    cout << PUTIH << "Masukkan nomor data yang akan diubah: " << KUNING;
    int index;
    cin >> index;
    cin.ignore(1000, '\n');
    cout << RESET;
```

Gambar 3.13 logic Update

```
if (index > 0 && index <= *jumlahLogistik) {

        // pointer arithmetic (Pointer ke elemen yang akan diubah)
        Logistik *ptrUpdate = logistik + (index - 1);

        cout << PUTIH << "Masukkan Komoditas baru: " << KUNING;
        getline(cin, ptrUpdate->mineral.namaMineral);

        cout << PUTIH << "Masukkan Jenis baru: " << KUNING;
        getline(cin, ptrUpdate->mineral.jenisMineral);

        cout << PUTIH << "Masukkan Satuan baru: " << KUNING;
        getline(cin, ptrUpdate->mineral.satuan);

        cout << PUTIH << "Masukkan Qty baru: " << KUNING;
        cin >> ptrUpdate->mineral.kuantitas;

        cout << PUTIH << "Masukkan Lokasi baru: " << KUNING;
        cin.ignore();
        getline(cin, ptrUpdate->asal.asalTambang);

        cout << PUTIH << "Masukkan Tanggal baru (dd-mm-yyyy): " << KUNING;
        getline(cin, ptrUpdate->tanggal);

        cout << "\n" << PUTIH << "    Status baru:" << endl;
        cout << KUNING << "    [1] Pending" << endl;
        cout << BIRU <<   "    [2] Shipped" << endl;
        cout << HIJAU <<  "    [3] Delivered" << RESET << endl;

        int statusPilihan = inputMenu(1, 3, "Pilih Status: ");
        switch (statusPilihan) {
            case 1: ptrUpdate->status = "Pending"; break;
            case 2: ptrUpdate->status = "Shipped"; break;
            case 3: ptrUpdate->status = "Delivered"; break;
        }
```

Gambar 3.14 logic update

```
cout << RESET;

        cout << HIJAU << "\n[OK] " << PUTIH << "Data logistik berhasil diubah" << RESET << endl;
    } else {
        cout << MERAH << "[!] " << PUTIH << "Nomor data tidak valid" << RESET << endl;
    }
    tekanEnter();
}
```

Gambar 3.15 logic update

H. Menghapus data Logistik

Menampilkan semua data logistik terlebih dahulu, lalu menghapus catatan sesuai indeks yang dimasukkan user.

Source Code:

```
void deleteLogistik(Logistik *logistik, int *jumlahLogistik) {
    CLEAR;
    cout << ABU_REDUP << "\n=========================================================" << endl;
    cout << EMAS      << "|                DELETE DATA LOGISTIK MINERAL            |" << endl;
    cout << ABU_REDUP << "=========================================================" << RESET << endl;

    if (*jumlahLogistik == 0) {
        cout << MERAH << "[!] " << PUTIH << "Belum ada data logistik untuk dihapus." << RESET << endl;
    } else {
        cout << ABU_TERANG << "--- [" << BIRU << " HAPUS DATA " << ABU_TERANG << "] ---" << RESET << endl;
        cout << ABU_REDUP << "====================" << RESET << endl;

        for (int i = 0; i < *jumlahLogistik; i++) {

            Logistik *ptr = logistik + i; // pointer arithmetic

            cout << PUTIH << "Data ke-" << EMAS << i + 1 << endl;
            cout << PUTIH << "ID: " << BIRU << ptr->idLogistik << endl;
            cout << PUTIH << "Komoditas: " << KUNING << ptr->mineral.namaMineral << RESET << endl;
            cout << endl;
        }

        cout << PUTIH << "Masukkan nomor data yang akan dihapus: " << KUNING;
        int index;
        cin >> index;
        cin.ignore(1000, '\n');
        cout << RESET;
        if (index > 0 && index <= *jumlahLogistik) {

            // Pointer arithmetic untuk menggeser elemen array
            for (int i = index - 1; i < *jumlahLogistik - 1; i++) {
                Logistik *ptrCurrent = logistik + i;       // pointer ke elemen saat ini
                Logistik *ptrNext    = logistik + (i + 1); // pointer ke elemen selanjutnya
                *ptrCurrent = *ptrNext; // dereference: salin isi struct
            }

            (*jumlahLogistik)--; // dereference pointer lalu decrement

            cout << HIJAU << "\n[OK] " << PUTIH << "Data logistik berhasil dihapus" << RESET << endl;
        } else {
            cout << MERAH << "[!] " << PUTIH << "Nomor data tidak valid" << RESET << endl;
        }
    }
    tekanEnter();
}
```

Gambar 3.16 Logic Delete

i. Tampilan Menu Utama

Menampilkan Menu utama setelah user berhasil login, Berisi berbagai pilihan fitur untuk mengelola logistik pertambangan.

Source Code:

```
void menuAdmin() {
    cout << ABU_REDUP << "\n\n=========================================================" << endl;
    cout << EMAS   <<    "|            MENU UTAMA (ADMIN)                          |" << endl;
    cout << ABU_REDUP << "=========================================================" << endl;
    cout << PUTIH  <<    "| No | Fitur Logistik                                    |" << endl;
    cout << ABU_TERANG <<"|----|-----------------------------------------------|" << endl;
    cout << PUTIH  <<    "| 1. | Tambah Data     (Create)                          |" << endl;
    cout << PUTIH  <<    "| 2. | Lihat Data      (Read)                            |" << endl;
    cout << PUTIH  <<    "| 3. | Ubah Data       (Update)                          |" << endl;
    cout << PUTIH  <<    "| 4. | Hapus Data      (Delete)                          |" << endl;
    cout << PUTIH  <<    "| 5. | Keluar ke Menu Login                              |" << endl;
    cout << ABU_REDUP << "=========================================================" << RESET << endl;
}
```

Gambar 3.17 Menu Utama

J. Fungsi main

Fungsi main merupakan titik awal program berjalan, didalamnya terdapat loop utama yang akan terus menampilkan menu awal hingga pengguna memilih untuk keluar. Jika pengguna berhasil login, maka akan diarahkan ke menu utama untuk mengakses fitur fitur utama.

Source Code:

```
int main() {

    #ifdef _WIN32
    SetConsoleOutputCP(65001);
    #endif

     // init users
    User users[MAX_USERS];
    int jumlahUser = 0;

    users[0] = {"rendy", "069", true}; // admin
    users[1] = {"rendi", "1234", false}; // user
    jumlahUser = 2;

    // init logistik
    Logistik logistik[MAX_LOGISTIK];
    int jumlahLogistik = 0;
    dataDummy(logistik, &jumlahLogistik); // init data dummy


    while (true) {
        CLEAR;
        asciiArt();
        displayMenuAuth();

        int auth = inputMenu(1, 3);

        if (auth == 1) {
            int idx = login(users, &jumlahUser);
            if (idx == -1) {
                exit(0);
            } else if (idx == -2) {
                continue;
            }
            mainMenu(users, logistik, idx, &jumlahLogistik);
        } else if (auth == 2) {
            registerUser(users, &jumlahUser);
        } else if (auth == 3) {
            CLEAR;
            cout << MERAH << "\n  Shutting down... " << PUTIH << "Program akan keluar." << RESET << endl;
            exit(0);
        }
    }

    return 0;
}
```

Gambar 3.18 fungsi main

4. Hasil Output

Gambar 4.1 Menu Auth

```
==============================================
||            LOGIN SESSION  # 1            ||
==============================================

  [>] Masukkan Username: rendy
  [>] Masuk Password: 069

[OK] Login berhasil. Selamat datang, rendy.

Tekan Enter untuk melanjutkan...
```

Gambar 4.2 Username dan Password benar

```
============================================================
|                       STATUS LOGIN                       |
============================================================
| Username : rendy                                         |
| Role     : ADMIN                                         |
============================================================


============================================================
|              MENU UTAMA (ADMIN)                          |
============================================================
| No | Fitur Logistik                                      |
|----|-----------------------------------------------------|
| 1. | Tambah Data     (Create)                            |
| 2. | Lihat Data      (Read)                              |
| 3. | Ubah Data       (Update)                            |
| 4. | Hapus Data      (Delete)                            |
| 5. | Keluar ke Menu Login                                |
============================================================

Pilih Menu (1-5):
```

Gambar 4.3 Menu Utama CRUD

```
==================================================================
--- [ TAMBAH DATA ] ---
==================================================================
ID Logistik            : LOG002 (otomatis)
Komoditas              : Emas
Jenis                  : Ore
Satuan                 : Kg
Qty                    : 6700
Lokasi                 : FreePort
Tanggal (dd-mm-yyyy)   : 06-07-2067
```

Gambar 4.4 Fitur Create

```
=====================================================
|              DAFTAR DATA LOGISTIK MINERAL          |
=====================================================
==========================================================================================================
| ID       | Komoditas          | Jenis         | Satuan  | Qty      | Lokasi                  | Tanggal          |
----------------------------------------------------------------------------------------------------------
| LOG001   | Bijih Nikel        | Laterit       | ton     | 100      | Tambang A - Sulawesi    | 01-01-2026       |
| LOG002   | Emas               | Ore           | Kg      | 6700     | FreePort                | 06-07-2067       |
==========================================================================================================

Tekan Enter untuk melanjutkan...
```

Gambar 4.5 Fitur Read

```
--- [ UBAH DATA ] ---
====================
Data ke-1
ID: LOG001
Komoditas: Bijih Nikel

Data ke-2
ID: LOG002
Komoditas: Emas

Masukkan nomor data yang akan diubah: 1
Masukkan Komoditas baru: Tembaga
Masukkan Jenis baru: ore
Masukkan Satuan baru: kg
Masukkan Qty baru: 1000
Masukkan Lokasi baru: Pit Mahakam
Masukkan Tanggal baru (dd-mm-yyyy): 01-03-2019

[OK] Data logistik berhasil diubah

Tekan Enter untuk melanjutkan...
```

Gambar 4.6 Fitur Update

```
--- [ HAPUS DATA ] ---
====================
Data ke-1
ID: LOG001
Komoditas: Tembaga

Masukkan nomor data yang akan dihapus: 1
```

Gambar 4.7 Fitur Delete

```
=============================================================
|           SISTEM LOGISTIK MINERAL PERTAMBANGAN            |
=============================================================
| No | Autentikasi                                          |
|----|------------------------------------------------------|
| 1. | Login Akun                                           |
| 2. | Register Akun Baru                                   |
| 3. | Keluar Program                                       |
=============================================================

Pilih Menu (1-3): 3

  Shutting down... Program akan keluar.
```

Gambar 4.8 Keluar Program

5. Langkah-langkah GIT

5.1 GIT Add

Digunakan untuk menambahkan semua perubahan (termasuk file baru, modifikasi, dan penghapusan) dalam direktori kerja ke staging area, yang berarti file tersebut siap untuk di- commit.

```
PS D:\vscode\Praktikum_APL_B2_25> git add Post-test/post-test-apl-4/
PS D:\vscode\Praktikum_APL_B2_25>
```

Gambar 5.1 git add

5.2 GIT Commit

Digunakan untuk menyimpan perubahan yang ada di staging area ke repository lokal, dengan menyertakan pesan commit yang menjelaskan perubahan tersebut.

```
PS D:\vscode\Praktikum_APL_B2_25> git commit -m "chore: upload post-test-4"
[main 10ad9f6] chore: upload post-test-4
 2 files changed, 606 insertions(+)
 create mode 100644 Post-test/post-test-apl-4/2509106069-Rendy-PT-4.cpp
 create mode 100644 Post-test/post-test-apl-4/2509106069-Rendy-PT-4.exe
```

Gambar 5.2 git commit

5.3 GIT Push

Digunakan untuk mengirimkan (upload) semua perubahan kode yang telah di commit dari repository lokal di komputer ke repository jarak jauh (remote)

```
PS D:\vscode\Praktikum_APL_B2_25> git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 20 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 74.65 KiB | 9.33 MiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/r3nddy/Praktikum_APL_B2-25.git
   af98502..6384222  main -> main
PS D:\vscode\Praktikum_APL_B2_25>
```

Gambar 5.3 git push